Task 1

(a) The F.conv2d function in PyTorch performs cross-correlation, where the kernel is applied directly to the image without flipping it. In true convolution, the kernel is flipped before applying it. The Gaussian blur acts as a low-pass filter because it smooths the image and reduces fine details and noise. Sobel-x and Sobel-y act as high-pass edge detectors, highlighting intensity changes in different directions while also providing some smoothing in the perpendicular direction. All three are 2D FIR filters that process an image using a small kernel.

(b) I used RandomHorizontalFlip and Random Affine for data augmentation. Horizontal flipping helps the model recognize garments even when their left and right sides are reversed. Random affine transformations with small rotations (±10°), shifts (10%), and scaling (0.9–1.1) help the model handle slight changes in the position, orientation, and size of clothing items. However, vertical flipping and large rotations are not suitable because clothes are not normally seen upside down, and rotating them too much can change their appearance and make different classes harder to distinguish.

(c) I split the 60,000 training images into 48,000 training images and 12,000 validation images using random_split with a fixed seed. The validation set helps me compare models and choose the best architecture and number of training epochs without using the test set. The test set is kept separate for the final evaluation so that the reported performance remains reliable. I applied data augmentation only to the training set because the validation set should represent the original data. Keeping validation images unchanged also makes the evaluation consistent across different epochs and models.

(d) get_device() in data.py.

Task 2 (default CNN: Conv32-ReLU-MaxPool-Conv64-ReLU-MaxPool-FC128-Dropout0.4-FC10)

Layer	In	k	s	p	Filters	Out	Params
Conv1	1×28×28	3	1	1	32	32×28×28	3·3·1·32+32 = 320
MaxPool	32×28×28	2	2	0	–	32×14×14	0
Conv2	32×14×14	3	1	1	64	64×14×14	3·3·32·64+64 = 18,496
MaxPool	64×14×14	2	2	0	–	64×7×7	0
Flatten	64×7×7					3136	0
FC1	3136				128	128	3136·128+128 = 401,536
Dropout(0.4)	128					128	0
FC2	128				10	10	1,290
Total							**421,642**

Task 2

(b) Verification of parameter count

I calculated the number of trainable parameters for each layer manually and compared the total with the value obtained using sum(p.numel() for p in model.parameters() if p.requires_grad). The total parameter count for the CNN is 421,642. This includes both weights and biases of the convolutional and fully connected layers.

(c) Receptive field

The receptive field represents the area of the input image that influences a particular neuron. In the first convolutional layer, the receptive field is 3 × 3 pixels. After the first max-pooling layer, it increases to 4 × 4 pixels, with a jump of 2 pixels. After the second convolutional layer, the receptive field becomes 8 × 8 pixels.

Since the original image is 28 × 28 pixels, this receptive field covers only a small part of the garment. Therefore, the convolutional layers mainly learn local features such as edges, textures, and small patterns. The fully connected layers combine these features to make the final classification.

(d) Training and overfitting

I monitored the training loss and validation loss throughout the training process to understand how well the model was learning. The epoch with the lowest validation loss is considered the best epoch based on this metric. If the training loss continues to decrease while the validation loss starts increasing after this point, it indicates that the model may be overfitting the training data.

(e) What did the filters learn?

The first convolutional layer learns different patterns from the input images. Some filters may detect edges and gradients similar to Sobel filters, while others may respond to blobs, contrast changes, or smooth regions. Unlike the hand-designed filters used in Task 1, these filters are automatically learned during training by minimizing the classification loss. Their actual functions can be understood by visualizing the learned kernels and their corresponding feature maps.

Task 3 — Ablation Study

(a) Convolution versus MLP

A CNN can perform better than an MLP on image classification because it uses local connections and shared weights to extract spatial features. An MLP processes the flattened image using fully connected layers, whereas a CNN can detect patterns such as edges and shapes more efficiently. The actual difference in validation accuracy should be used to determine which model performed better in this experiment.

(b) Effect of pooling

Pooling reduces the spatial dimensions of feature maps, which decreases the number of inputs passed to the fully connected layers and can reduce computation. It also helps the network become less sensitive to small changes in feature positions. A stride-2 convolution can reduce spatial dimensions similarly, but it uses learnable weights. The validation accuracy, parameter count, and training time from the experiment help compare the three approaches.

(c) Effect of network depth

Increasing the number of convolutional blocks can help the model learn more complex features. However, adding more layers does not always improve performance. For 28 × 28 images, repeated downsampling can reduce the spatial resolution too much, while deeper networks can become harder to optimize and may overfit the training data. Therefore, the best depth should be selected based on the validation results.

(d) Effect of data augmentation

Data augmentation introduces small variations into training images, helping the model learn features that generalize better to unseen data. It may increase training loss because the model sees more varied examples, but it can reduce the gap between training and validation performance. Whether it improves validation accuracy depends on the actual experimental results.

Task 4 — Transfer Learning with ResNet-18

(a) Input preprocessing

ResNet-18 was pretrained on ImageNet, so I used the ImageNet mean and standard deviation to normalize the input images. This helps keep the input distribution consistent with the preprocessing used during pretraining, allowing the model to make better use of its learned features.

(b) Model setup

Fashion-MNIST images were resized from 28 × 28 to 64 × 64 pixels and converted from grayscale to three channels. The final fully connected layer of ResNet-18 was replaced to produce predictions for the 10 Fashion-MNIST classes. The AdaptiveAvgPool2d((1,1)) layer allows the network to convert the final feature maps into a fixed-size representation before classification, making it possible to use 64 × 64 inputs.

(c) Comparison of fine-tuning strategies

Feature extraction freezes the pretrained backbone and trains only the new classification layer. This approach requires fewer trainable parameters and generally trains faster, but its performance may be limited because ImageNet features are not perfectly suited to Fashion-MNIST images.

Gradual unfreezing allows the final convolutional block and classifier to adapt to the new dataset. Differential learning rates allow the entire network to be fine-tuned while using a smaller learning rate for the pretrained backbone than for the new classifier. These approaches may improve validation accuracy, but they require more computation and can overfit when training data is limited.

The best strategy should be selected by comparing validation accuracy, trainable parameters, and time per epoch using transfer_summary.csv.

(d) Learning-rate scheduler

Cosine annealing gradually reduces the learning rate during training. This allows the model to make larger updates earlier and smaller adjustments later. During transfer learning, a lower learning rate toward the end can help preserve useful pretrained features while allowing the model to adapt to the new dataset. Its effectiveness should be evaluated by comparing the validation curves with those from constant learning-rate training.

Task 5 — Final Evaluation and Complexity Analysis

The custom CNN and ResNet-18 should be evaluated on the official test set after selecting the best models using validation accuracy. Test accuracy and macro-F1 score provide complementary measures of classification performance.

Some clothing classes, such as Shirt and T-shirt, or Pullover and Coat, may be difficult to distinguish because their shapes and details can look similar at 28 × 28 resolution. However, the most frequently confused pair should be identified from the actual confusion matrix.

The custom CNN has approximately 0.42 million parameters, whereas ResNet-18 has approximately 11 million. If both models achieve similar test performance, the smaller CNN may be a more efficient choice because it requires fewer parameters and potentially less training time. The final conclusion should be based on the measured accuracy and runtime in efficiency_table.csv.

Task 6 — Explainability with Grad-CAM

(a) Implementation

I used model.layer4[-1] as the target layer for Grad-CAM. This layer provides the feature maps from the final residual block after its batch normalization and ReLU activation. Since the input images are 64 × 64 pixels, the resulting feature map is only 2 × 2. Consequently, the Grad-CAM heatmap has limited spatial detail and must be resized to overlay it on the original image.

(b) Interpretation of correctly classified images

For the correctly classified Coat and Shirt images, the Grad-CAM heatmaps can be used to identify the regions that contribute most strongly to the model's predictions. Attention to garment features such as sleeves, collars, hems, and overall shape may indicate that the model is using relevant visual information. If the heatmaps highlight the background or unrelated regions, the model may be relying on less meaningful features.

(c) Failure-case analysis

For a misclassified image, the Grad-CAM heatmap helps investigate which regions influenced the incorrect prediction. The model may have focused on an ambiguous part shared by two clothing classes, such as a similar neckline or silhouette. Alternatively, it may have focused on irrelevant background features. Comparing the heatmap with the confusion matrix can help explain the possible reasons for the classification error.

(d) Limitation of Grad-CAM

Grad-CAM provides a visual indication of the image regions that influence a particular class prediction, but it does not fully explain the model's decision-making process. A heatmap that appears reasonable does not prove that the model relies on the intended features, because it may also use correlated background information or other unintended patterns.

Task 7 — Discussion

(a) Effect of data augmentation

Data augmentation helped the model learn from images with small variations in position, orientation, and size. It can reduce overfitting by improving the model's ability to generalize to unseen images. The effect on the training–validation gap and final validation accuracy should be confirmed from the results obtained in Task 3(d).

(b) Best transfer-learning strategy

The best transfer-learning strategy should be selected based on the validation accuracy obtained in Task 4(c). Unfreezing the final convolutional block or using differential learning rates may improve performance because the pretrained features can adapt to Fashion-MNIST. However, the actual results are needed to determine which strategy performed best.

(c) Most frequently confused classes

Classes such as Shirt, T-shirt, Pullover, and Coat can be difficult to distinguish because they share similar shapes and silhouettes. At the low resolution of 28 × 28 pixels, fine details such as collars, sleeves, and buttons may not be clearly visible. As a result, the model may rely more on the overall shape of the garment than on its finer details.

(d) Did Grad-CAM highlight meaningful regions?

Grad-CAM can help identify the regions of an image that contributed to the model's prediction. If the heatmaps highlight important garment features, such as the sleeves, collar, or overall shape, it suggests that the model is focusing on relevant regions. The final observation should be based on the visualizations obtained in Task 6(b).

(e) What did the failure-case visualization reveal?

The failure-case visualization helps us understand which image regions influenced an incorrect prediction. For example, the model may have focused on a region shared by two similar clothing classes or on irrelevant background details. This provides additional

insight into the possible reasons for misclassification beyond what the confusion matrix alone can show.

(f) Comparison between a learned convolutional filter and the Sobel filter

The first convolutional layer of a CNN performs a filtering operation similar to a Sobel filter by applying a small kernel to the input image to detect patterns such as edges and intensity changes. However, Sobel filters have fixed, manually designed values, whereas CNN filters are learned automatically during training. The 32 filters in the first convolutional layer can learn different features, including edges, textures, and blobs. These learned features are then combined by deeper layers to recognize more complex patterns, allowing the CNN to adapt its filtering operations to the classification task.